



AI 网站克隆模板 小白指南

丢一个网址给 AI，它把整个网站"一对一复刻"成干净的 Next.js 代码

★ 35.6k stars

GitHub: JCodesMore/ai-website-cloner-template

v0.5.1 • MIT

来源：拾遗队列推文

一、它到底是什么？（疑问 1、2）

Q1 它是"模板"，不是软件库——交付的到底是什么？

传统开源库你 `npm install` 后调用它；这个项目反过来——你安装的是"给 AI 编程智能体看的作业指导书"，真正干活的工人是 Claude Code / Cursor 这些 AI。打开仓库你会发现核心资产只有三样：

资产	大小	作用
<code>.agents/skills/clone-website/SKILL.md</code>	506 行	五阶段克隆流水线的完整提示词（核心中的核心）
<code>.agents/skills/.../inspection-guide.md</code>	80 行	侦察阶段的检查清单（截什么图、量什么参数）
Next.js 16 脚手架	src/ + public/	预搭好的空 Next.js 项目，克隆结果写进这里

一句话：它 = 506 行的"方法论提示词" + 一个能编译的空项目骨架。这正是我们知识库里的 [[AgentSkills技能包]] 概念的顶级实战案例——把工程经验封装成 SKILL.md 喂给智能体。

Q2 直接对 Claude 说"帮我复刻这个网站"不行吗？模板多给了什么？

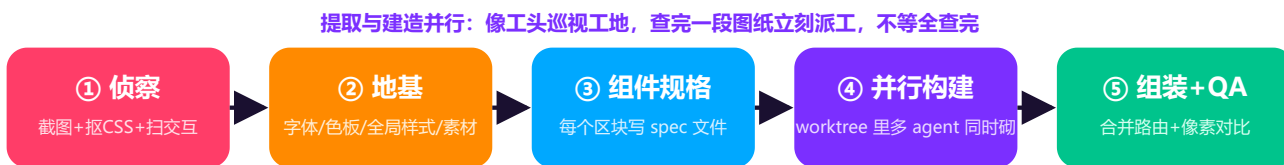
不行，差得很远。直接说一句话，AI 会"凭感觉"做：颜色靠猜、间距靠估、交互靠脑补，做出来是"形似的假货"。模板把手工经验固化成了纪律：

- **不许猜**：所有样式值必须用 `getComputedStyle()` 从真实网页抠出来，"看起来像 text-lg"这种估算直接判违规；
- **不许一口吞**：整个页面不许交给一个 agent 一次做完，必须切块、写规格书、分派给并行的 builder；
- **不许半成品**：每一步合并后 `npm run build` 必须绿，"临时坏一下"都不行；
- **不许只抄静图**：网站是活的——滚动、悬停、点击的每个状态都必须抓下来。

这就是把"老师傅的手感"翻译成"新工人能执行的工序卡"，是 [[构建流水线与Pass]] + [[质检Gate与自我纠错循环]] 两个已有概念在提示词层的落地。

二、五阶段流水线怎么运作？（疑问 3、4、5）

Q3 总流程：一条流水线，五个阶段



关键设计：**这不是“先全部侦查、再全部施工”的两段式，而是流水线**——工头（主 agent）每侦察完一个区块，马上写规格书、派 builder，自己继续往下侦察。builder 在后台施工的同时，工头的侦察不停，总工期大幅缩短。

Q4 为什么每个组件必须先写 spec 文件？精确值为什么不能目测？

spec 文件（`docs/research/.../components/xxx.spec.md` 侦察工与施工 agent 之间的“合同”，内容全部来自真实测量的 `getComputedStyle()` 输出：

```
// 每个元素抠出 40+ 个属性，精确到像素
fontSize: "16px"   fontWeight: "600"   lineHeight: "24px"
padding: "12px 24px"   borderRadius: "9999px"
boxShadow: "0 4px 20px rgba(0,0,0,0.1)"
transition: "all 0.3s ease"
```

为什么不能目测？**18px 和 text-lg 看着一样，但 line-height 可能差 4px，整页累计就是“神韵不对”**。spec 同时是可审计的产物——克隆走样时，回头查 spec 是“侦察量错了”还是“施工抄错了”，责任可定位。这与知识库 [[确定性脚本]] 思想同源：主观感受不可复现，客观测量才能当合同。

Q5 “先滚动观察、再点击测试”——最贵的坑

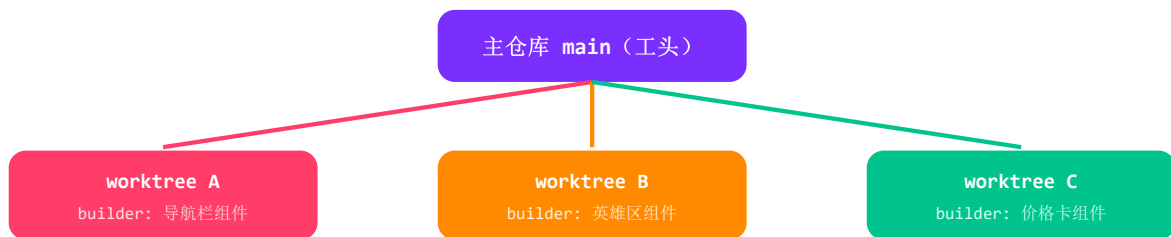
SKILL.md 用加粗红字警告：**克隆失败最昂贵的错误，是把滚动驱动的区块建成点击驱动的**——这不是改几行 CSS 能救的，是整个组件推倒重写。所以工序强制：

1. 先别点！慢慢滚过该区块，观察有没有东西自己动（导航栏变小？卡片浮入？侧栏高亮自动切换？）；
2. 若有 → 判定为滚动驱动，抓取机制（`IntersectionObserver` / `scroll-snap` / `position:sticky`）；
3. 确认滚动无反应 → 再用点击/悬停测试；
4. 把结论写死进 spec：`INTERACTION MODEL: scroll-driven` 或 `click-to-switch`。

另外每个组件要抓**所有状态**而非默认态：标签页要每个都点一遍抓内容，导航栏要抓“滚动前/后”两组 CSS 再做差分——差分就是行为说明书。

三、并行构建与命名隔离（疑问 6、7）

Q6 git worktree: 多个工人同时施工怎么不打架？



各自独立目录施工 → 互不覆盖 → 完成后逐个 merge 回 main, 每次合并跑一次 build 验证

如果三个 builder 都在同一个目录写文件, 同时改 `globals.css` 就是灾难。git worktree 让每个 builder 拿到仓库的独立工作副本（独立分支+独立目录），完工后工头逐个合并、每次合并立即验证编译。这是“小任务、完美结果”原则的工程保障。

配套纪律是 150 行复杂度预算：一份施工说明书超过约 150 行规格，说明区块太复杂，必须再切小。因为拿到“整个 features 区”的 agent 会偷懒近似，拿到“一张卡片 + 精确 CSS”的 agent 每次都完美交付。

Q7 site-key / page-key 里的 SHA-256 是干什么的？

每克隆一个站点，产物都装进带“指纹”的命名空间，例如：

```
site-key = nextjs-org + SHA256(网址)前8位 → "nextjs-org-a1b2c3d4"
产物路径: docs/research/<site-key>/<page-key>/components/hero.spec.md
public/sites/<site-key>/<page-key>/images/logo.webp
```

目的：防止克隆第二个网站时覆盖第一个网站的研究资料、截图和素材。可读前缀让人看得懂，哈希后缀保证“github.com”和“gitee.com”这类相似来源绝不撞车——同一套工程里可以安全积累多个克隆作品。

四、多 AI 工具兼容与工程纪律（疑问 8、9）

Q8 一份技能包，四家 AI 工具都能用——“薄桥接”设计



工作流只写一份（`.agents/`），Cursor/Codex/OpenCode 天然认这个路径；Claude Code 走自己的 `.claude/commands/`，但那里只有 6 行一读规范、执行，没了。好处：改流程只改一处，四家工具同步生效，永不出“两个版本说法打架”。这是典型的 DRY 原则在 agent 配置层的应用。

Q9

为什么把"每次合并必须编译通过"写死成铁律？

SKILL.md 尾部有一节"What NOT to Do"，全是 从失败克隆里用血泪换来的禁令（作者原话：每条都花掉过几小时返工）。核心三条：

- **Build Must Always Compile**: 并行施工最怕"烂尾楼合流"—每个 builder 完工前必须 `npx tsc --noEmit` 通过，每次合并后必须 `npm run build` 通过，仓库任何时刻都是可运行状态；
- **Builder 提示词**必须内联完整规格：不许说"参考 DESIGN_TOKENS.md"，工人必须拿到全部信息，零猜测；
- 不许把新目标当"推倒重来"的许可：克隆第二个站不得碰第一个站的任何路由与产物。

这和我们知识库 [[质检Gate与自我纠错循环]] 的结论完全一致：AI 生成的代码必须由 客观的确定性裁判（编译器）把关，而非"看起来对"。

五、跑起来：条件、边界、验收（疑问 10、11、12）

Q10

完整跑一次克隆需要什么？

条件	说明
Node.js ≥ 24	本机 v24.21.0 ☒
AI 编程智能体	Claude Code（官方推荐配 Opus） / Cursor / Codex CLI / OpenCode
浏览器自动化 MCP	Chrome MCP / Playwright MCP 之一— <u>没有它流程直接拒绝启动</u> ，因为侦察阶段全靠真开浏览器量 CSS
使用方式	GitHub 点 "Use this template" 建自己的副本（不要直接 clone 原仓库开工）→ 装依赖 → 在项目里输入 <code>/clone-website https://目标.com</code>

范围默认：只克隆前端视觉与交互（布局/样式/动效/响应式 + 假数据），不含真实后端、数据库、登录、实时功能。

Q11

法律与伦理边界（README 白纸黑字）

 **明确禁止**：钓鱼/冒充网站；盗用 logo、品牌资产（归原主所有）；违反目标网站服务条款的行为。

正当用途：把 WordPress/Webflow 老站迁移成现代 Next.js 代码库；找回丢失源码的自有网站；学习生产级网站的布局、动画、响应式实现手法（对学习价值最大）。一句话：学手法可以，冒名顶替不行。

Q12

"像素级"怎么验收？—Phase 5 视觉 QA 对比

组装完不许宣告完工。最后一步是把 原站和克隆站并排放，1440px 桌面 + 390px 手机逐区块对比截图，发现差异就回溯：先查 spec 文件—侦察量错了就重测更新规格，量对了就是施工错了、改组件。然后真人式测试全部交互：滚一遍、每个按钮点一遍、悬停全试一遍。全部通过才算交付。最终报告要交代：区块数、组件数、spec 数（必须与组件数相等）、素材下载数、build 状态、遗留差异。

六、这个项目给我什么（能力清单）

迁移老站 找回丢源码的站 学习大厂前端手法 快速生成改版底稿 研究竞品交互模型

- 学手法：把喜欢的网站克隆下来逐行读代码，比看教程快十倍——这是对学习者最实在的用法；
- 学方法论：这份 506 行 SKILL.md 本身就是“如何给 AI 写工程级提示词”的范文（规格先行、Gate 把关、并行隔离）；
- 迁移与救急：老平台网站转 Next.js、丢失源码的站重建；
- 二次开发：克隆只做“纯仿制”，之后随便改造成自己的版本（注意品牌资产边界）。

七、动手实验（本机全部实测通过）

实验 1：把模板脚手架跑起来（主实验）

第1步 克隆仓库：`git clone https://github.com/JCodesMore/ai-website-cloner-template repo` — 拿到 v0.5.1 (f50066d)。

第2步 装依赖：`cd repo && npm install` → ✓ 退出码 0，node_modules 生成。

第3步 质检三连：`npm run check` (lint + typecheck + build) → ✓ 退出码 0，真实输出：

```
✓ Compiled successfully in 9.0s
Finished TypeScript in 1391ms
✓ Generating static pages using 5 workers (4/4) in 610ms
```

第4步 起服务验证：`npm start` 后 `curl http://localhost:3000` → ✓ HTTP 200、6727 字节，返回标准 Next.js 首页（Geist 字体 preload + App Router 骨架）。

✓ 预期结果：四步全绿，说明“空脚手架”完全可用——真实克隆就是在它上面长出组件。

实验 2：亲手写一个“侦察阶段”脚本（recon.mjs）

浏览器 MCP 我们暂时没有，但侦察的本质可以先用 `fetch + 正则` 演示：清点目标网页的 img/video/背景图/内联 SVG/字体/favicon，并识别技术栈指纹。脚本在 `exercise\recon.mjs`，运行 `node recon.mjs <url>`。实测两组输出：

```
$ node recon.mjs https://example.com # 对照组
→ 713 字节, 0 图 0 SVG 0 视频 (史上最素网页)

$ node recon.mjs https://nextjs.org # 实验组
→ HTTP 200, 366KB HTML, 65 个 <img>, 77 个内联 SVG
→ isNextJs = true (发现 /_next/ 指纹), Tailwind 类名命中 267 处
```

✓ 预期结果：同一脚本对“素人网页”和“生产级网页”产出数量级差异的资产清单——这正是流水线第①阶段要“看清楚再动工”的原因。实验中还修了一个真 bug: `String.match` 带全局标志会忽略捕获组，必须用非全局正则另取组。

说明：完整五阶段克隆（含截图、getComputedStyle 取值、并行 builder）需要浏览器 MCP + Claude Code 交互环境，本实验聚焦“验证脚手架可跑 + 侦察原理可复现”，流水线其余环节通过精读 506 行 SKILL.md 掌握。

一句话总结

这个项目证明，**AI** 的输出质量上限，取决于你给它的工序纪律下限。506 行提示词把"网站复刻"从碰运气的手艺活，变成了侦察有清单、施工有图纸、合并有 **Gate**、交付有验收的工业流水线——而这四件事，正是我们知识库里 **[[构建流水线与Pass]]**、**[[质检Gate与自我纠错循环]]**、**[[AgentSkills技能包]]**、**[[多智能体协作]]** 四个概念的一次完美合体演练。

生成于 2026-10-02 · F:\reminder\ai-website-cloner · 拾遗队列 task 1787578514459 · 数据来源: GitHub 仓库实测 + README/SKILL.md 精读